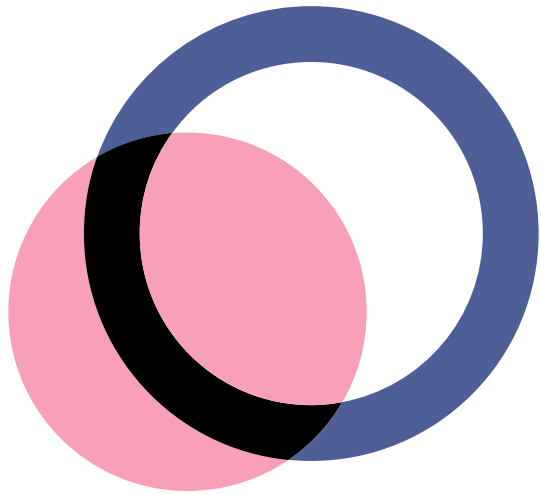
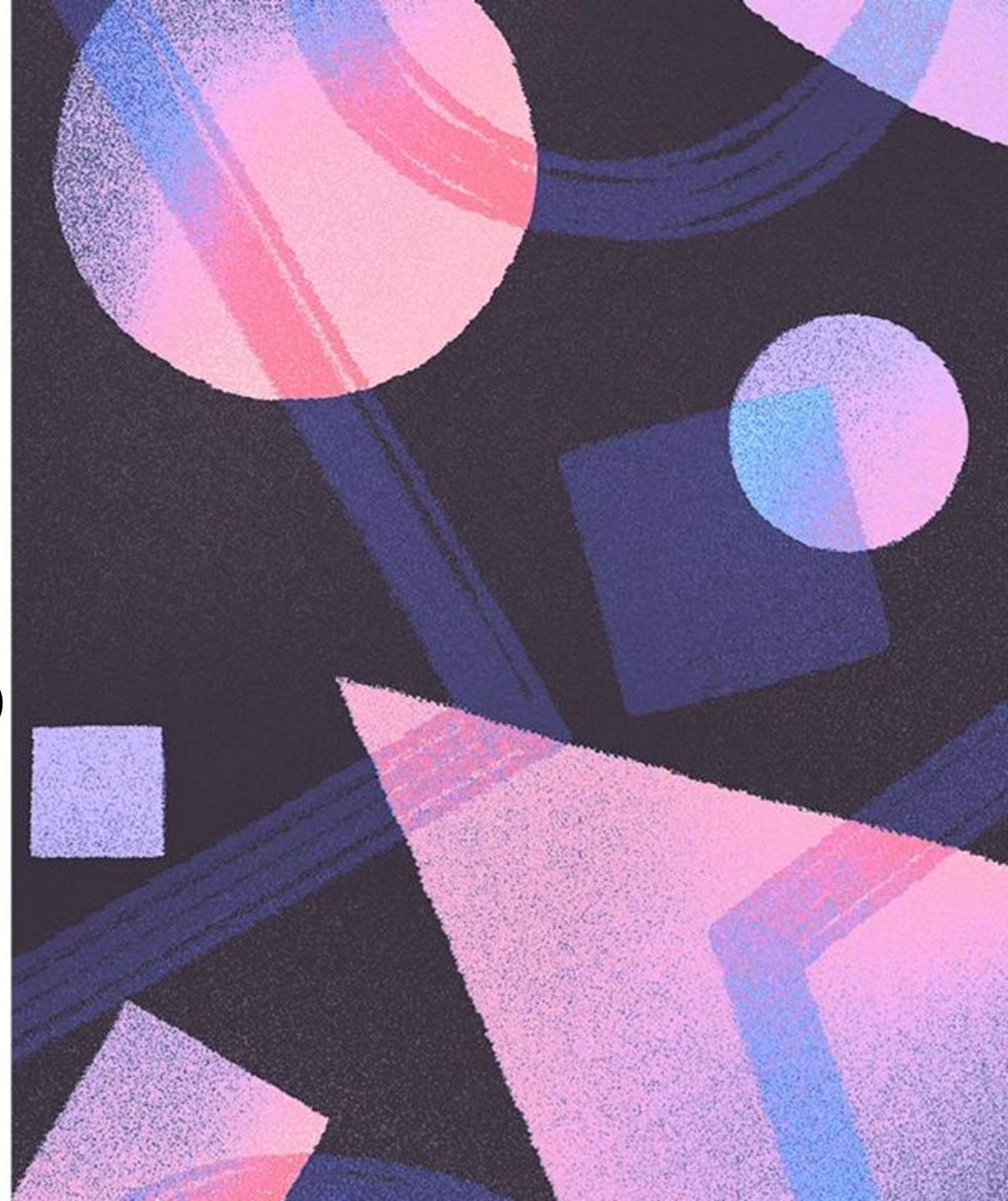




Design Patterns Lab

Lecture 7

> Strategy Pattern



What is Strategy?

- Part of Behavioral Design Patterns (BDP)
- Focus: Dynamic behavior selection at runtime
- Core idea: Interchangeable algorithms using composition

Transition from Template Method

- Template Method uses inheritance
- Behavior defined inside subclasses
- Behavior becomes fixed after object creation
- Limitation: cannot change behavior dynamically

Problem Statement

- Systems require flexible and changeable behavior
- Multiple algorithms solve the same problem differently
- Need to switch between them at runtime
- Avoid creating many subclasses

Scenario: University Submission (Scoring)

- System calculates scores for submissions
- Different scoring strategies exist:
 - Computer Science scoring
 - Design scoring
 - Business scoring
- Requirement: switch scoring logic dynamically

Strategy Pattern Idea

- Define a family of algorithms
- Encapsulate each algorithm in a separate class
- Make them interchangeable
- Inject behavior into a context object

Core Design Structure

- Strategy Interface → defines common behavior
- Concrete Strategies → implement different algorithms
- Context Class → uses a strategy object
- Responsibilities split:
 - Strategy → defines behavior
 - Context → executes behavior

Key Concept: Composition

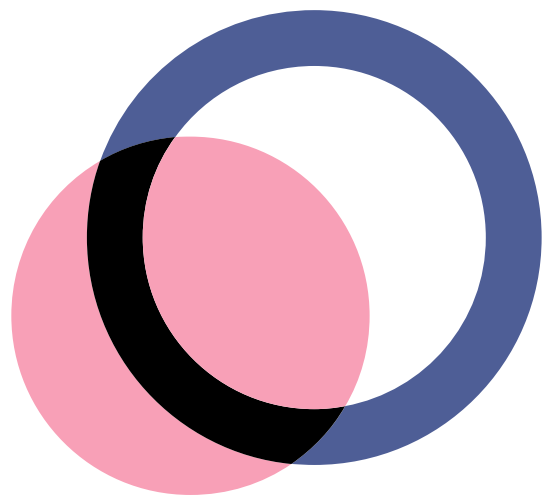
- Context “has a” strategy
- Not based on inheritance
- Behavior is delegated, not implemented directly
- Enables flexible design

Key Benefits

- High flexibility and extensibility
- Eliminates conditional logic (if/switch)
- Promotes Open/Closed Principle
- Reusable and independent algorithms

Activity: Dynamic Payment System

- System processes payments using different methods
- Payment methods: Credit Card, PayPal, Cash
- Requirement:
 - Switch payment method at runtime
 - Use same context object
 - Extend system with new payment methods easily



**THANK
YOU**

